SUPPLEMENTARY MATERIAL

MeshDoc

Offline first, end to end encrypted collaboration for teams without reliable connectivity.

Most collaboration software assumes a server. MeshDoc assumes none, and treats the internet as an optional enhancement. An Android phone holds structured documents (checklists, supply forms, location notes) that are encrypted before they leave the device, synchronised directly with other phones over Bluetooth and Wi-Fi Direct, and passed to an internet relay when one happens to be reachable. When two phones meet, they reconcile and converge. When one climbs a hill and finds signal, the whole team's state reaches headquarters.

This appendix is evidence, not argument. It documents a working prototype built before the application, so that a reviewer can judge the plan against something that runs rather than against a description of something that might. It also sets out the architecture, the security model and its limits, the budget and the risks.

133	2	MIT	4
automated tests passing	Android devices converged	licence, no exceptions	Rust crates planned

Applicant	Sushant, Arvion Technologies, Kathmandu, Nepal
Fund	NLnet Restack, Open Internet Stack
Requested	29,920 euro over twelve months
Source	`https://github.com/sushant-me/meshdoc`
Licence	MIT
Date	2 November 2026

What is in here. Section 1 is what exists today and how it is verified. Section 2 is the architecture. Section 3 follows one edit from a tap to a converged replica. Section 4 is the security model, including what it does not protect. Section 5 is the plan and the money. Section 6 is the risk register. Section 7 is prior art.

1. What exists today, and how we check it

This is a prototype, not a product. We built it to answer one question before asking anyone for money: are the hard parts actually as hard as they look? The table below is blunt on purpose, including where things are only half done.

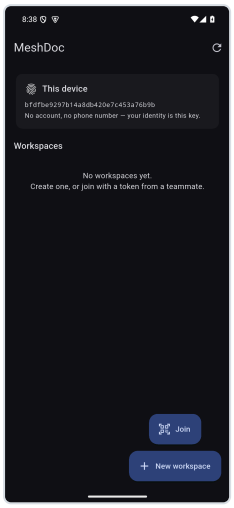
Component	State	Evidence
Device identity	**Working**	Ed25519 signing plus X25519 agreement, generated on device, never transmitted. No accounts, no phone numbers.
Workspaces and membership	**Working**	Signed, append only membership log with key rotation epochs. A non member holding the workspace key still cannot have a change applied.
Encrypted change envelope	**Working**	XChaCha20 Poly1305, with workspace, document, epoch and author bound as associated data. Relabelling an envelope into another document fails.
CRDT documents	**Working**	Four typed views over an existing CRDT: checklist, supply form, geo point, note. Concurrent edits converge.
Anti entropy sync	**Working**	Peers exchange content addresses and transfer only the difference, in pages, resuming after a dropped link without resending.
Relay and delay tolerance	**Working**	Opaque blob store with sequence numbers. Two peers that are never online together still converge through it.
BLE discovery	**Implemented**	Rotating 128 bit service UUID derived from the workspace key, 15 minute bucket, unlinkable to observers. Advertising verified running on Android 15. Not yet tested phone to phone.
Wi-Fi Direct formation	**Implemented**	Deterministic owner election from the two identity fingerprints, so both sides always request opposite intents and negotiation cannot stall. Authenticated handshake before any sync. Not yet tested on physical hardware.
Android client	**Working**	Flutter interface, Rust core over FFI, Kotlin radios. Installs, launches, creates documents, advertises, scans, syncs.
Two device convergence	**Verified**	Two Android 15 devices, syncing in both directions with edits made while apart, ending byte identical.

Automated test coverage

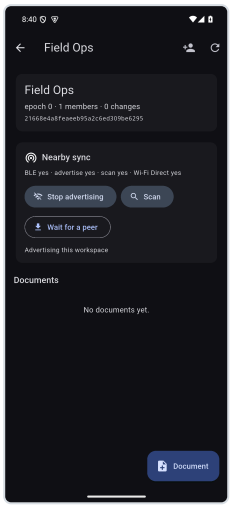
Suite	Tests	What it covers
Core library	76	Cryptography, membership authorisation, document convergence, sync protocol, transport framing, encrypted store, BLE tag derivation, group role election
Convergence fuzzing	7	Seeded random edit and partition schedules over 3 to 6 replicas, then a full heal, asserting identical state and no lost work
Relay client	30	HTTP framing, pagination, timeouts, error surfaces
Relay server	15	Idempotency, crash recovery, concurrent writers, quota and eviction
FFI boundary	5	The C ABI the Android app uses, including a full two store sync driven only through the exported functions
Total	**133**	

1.1 The client, running

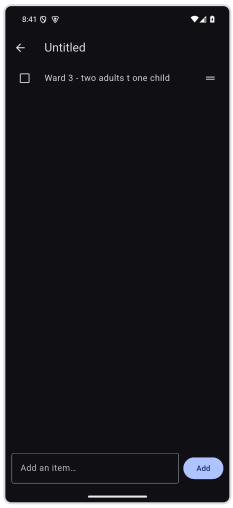
Screenshots from an Android 15 (API 35) device. The identity on the first screen is generated by the Rust core inside the app. The second shows the nearby sync panel with the capabilities the Kotlin layer actually reports, and BLE advertising running. The third is a checklist, and every tick on it becomes an encrypted change.



Identity made on device. No account, no registration.



Discovery panel, advertising active.



Large targets, for gloved hands.

What the client does today

- Create a workspace, or join one from an invite token.
- List documents with a live summary per type.
- Tick checklist items and add new ones.
- Advertise the rotating discovery tag over BLE.
- Scan for peers on that same tag.
- Form a Wi-Fi Direct group and sync with a peer.
- Pull from a relay over the internet.

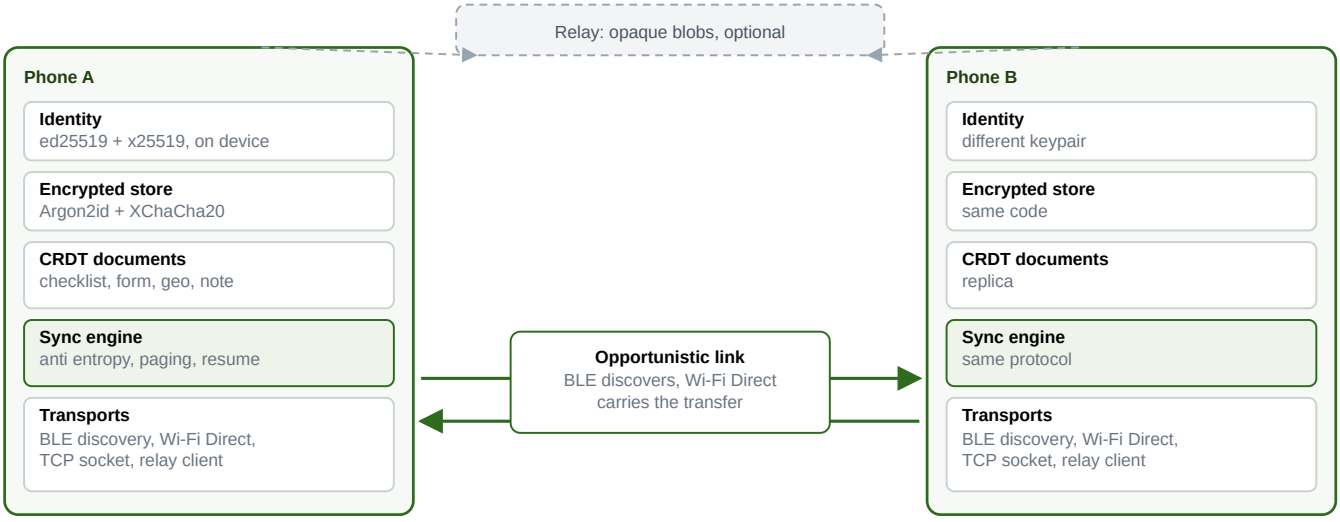
When something native fails, the app says so in words instead of crashing. In the field a silent failure is worse than a loud one.

> **The claim we are not making.** The radio transports are written and their logic is unit tested, but no two physical phones have exchanged a packet over BLE or Wi-Fi Direct yet. Everything below the radio link is proven. The radio itself is the work the grant would pay for.

We would rather write that down here than have a reviewer discover it later.

2. Architecture

Two equal peers. There is no master, no leader and no always on server. The only optional component is the relay, and it is a post office: it stores sealed envelopes and hands them out later.



The relay stores ciphertext it cannot read. It is never required for correctness.

Five things the design will not compromise on

Invariant	Why it is there
Every wire format is encrypted before it is transported	The mesh, the relay, and a stolen phone's disk all look the same to an attacker: ciphertext.
No server is required for correctness	The relay can be offline, hostile, or gone. Convergence happens by meeting.
Every change is attributable to a key in the membership log	Holding the workspace key lets you read. It does not let you write, because writes are authorised by a signed log rather than by a shared secret.
The workspace identifier is never derived from a key	Rotating a key must not change the identity of the workspace, or every stored reference and relay mailbox would break.
A document is the unit of merge, not a database row	Field teams edit a checklist, not a table. Shipping CRDT documents keeps merge semantics honest.

> **Why a trait for transport.** The sync engine is written against a narrow transport interface: advertise, discover, connect, send, receive. BLE discovery, Wi-Fi Direct, a plain socket and the relay are all implementations. That is what makes the radio work replaceable if a vendor stack misbehaves, and what lets other projects reuse the transport crate without adopting the rest.

3. The life of one edit

Following a single tick of a checkbox from a tap to a converged replica on another phone, with the numbers the prototype actually measures.

#	Step	What happens
1	Local edit	The CRDT updates in memory and produces a delta. Nothing has left the device.
2	Encode	The delta is sealed with the workspace epoch key. Workspace id, document id, epoch and author are bound as associated data, so the ciphertext cannot be relabelled.
3	Sign	The author signs the header plus a hash of the ciphertext with their Ed25519 device key. A change is attributable forever.
4	Store	The envelope is written to the encrypted local store and appended to the change log. A typical checklist tick is around 400 bytes on the wire.
5	Discover	The phone advertises a 128 bit service UUID derived from the workspace key and a 15 minute time bucket. Only members can compute the sequence, so outsiders see an unrelated random identifier that rotates.
6	Connect	On a match, the two phones elect a Wi-Fi Direct group owner deterministically from their fingerprints, so both sides request opposite roles and negotiation cannot stall. Each side then proves possession of a member key before any sync traffic.
7	Reconcile	Peers exchange the content addresses they hold. Only what is missing crosses the link, in bounded pages. A link that dies mid transfer resumes on the next contact with nothing resent.
8	Verify and apply	Signature, membership at the claimed epoch, and AEAD all check out before the CRDT integrates the delta. A forged change is stored but never applied.
9	Converge	Both replicas reach byte identical state without a conflict dialogue, because the data type defines the merge rather than a server deciding.
10	Relay, if there is signal	Any node may push the same envelopes to a relay as opaque blobs. A peer that is never online at the same time pulls them later. Direct contact is always preferred when available.

Why gossip sync works in the field

Sync is gossip, not broadcast. If A meets B and B later meets C, then C has A's changes. The network repairs itself through human movement, which is how field teams actually work. This also means the system degrades gracefully: a team of five does not need five phones in one place, only a chain of contacts.

Change size
`~400 bytes`
checklist tick on the wire

Transfer frame
`4,096 bytes`
bounded, resumable chunk

Tag rotation
`15 minutes`
plus two bucket skew tolerance

4. The security model, and what it does not protect

A threat model that only lists strengths is marketing. This one lists the edges as plainly as the guarantees, so a reviewer can tell exactly what a field deployment would and would not buy.

Threat	Protected	Mechanism, or why not
Relay operator reads content	**Yes**	Blobs are sealed with the workspace key before upload. The relay stores and returns bytes it cannot interpret.
Network observer on the internet	**Yes**	Content is encrypted end to end, so TLS is a second layer rather than the only one.
Stolen or seized phone, at rest	**Yes**	The local store is sealed with a key stretched from the passphrase using Argon2id. Without the passphrase the change log is opaque.
Non member injects a change	**Yes**	Writes are authorised by the signed membership log, not by possession of the workspace key. A stolen key reads but does not write.
Network observer of BLE discovery	**Partly**	No stable application layer identifier: the UUID rotates every 15 minutes and only members can derive it. Link layer tracking is outside what an app can fix, and we say so.
Relay learns metadata	**No**	The operator sees a workspace pseudonym, blob sizes and upload timing. Mailbox identifiers rotate, which limits but does not remove correlation.
Malicious member	**No**	A member can read everything and sign valid changes. Detecting that is a social problem, not a cryptographic one.
Removed member reads new data	**Partly**	Rotation stops them reading content written under a later epoch. It cannot un read what they already saw, and a partitioned member keeps using the old epoch until they learn of the rotation.
Traffic analysis and link layer tracking	**No**	Not claimed. Doing it properly needs cover traffic, which conflicts with the energy target.
Device loss with no passphrase	**No recovery**	There is no key escrow, by design. Losing the passphrase means losing the local data.

> **The rule we hold ourselves to:** if a claim cannot be backed by a measurement or a construction, it does not go in the proposal. That is why the energy target is written as a target with a measurement method attached, and why the radio transports are described as implemented but not yet demonstrated on physical hardware.

Membership without a server

A workspace key is easy to share and impossible to revoke cleanly, so possession alone cannot be the authorisation rule. Instead each workspace carries a signed, append only log of membership entries, and a change is accepted only if its author appears in the receiver's own view of that log at the epoch the change claims. Because the log is a hash chain, entries can arrive out of order and still verify. Concurrent additions are allowed, and removals carry the epoch they take effect in.

Longer term the group key layer moves to MLS (RFC 9420), which is the right answer for a workspace with many members and frequent churn. The prototype is deliberately simpler than that, and the design documents say so.

5. Plan and budget

Where the money goes

WP	Work	Days	Euro	Deliverable
WP1	Setup, architecture, threat model	9	1,530	Repository, licence, CI, decision records, written threat model
WP2	Protocol core, crypto, document bridge	34	5,780	Four document types, change format, identity, epochs, rotation, schema migration
WP3	Transport abstraction and fast path	20	3,400	Transport trait and a conformance suite any implementation can run
WP4	Free software transports	26	4,420	BLE discovery with rotating identifiers, Wi-Fi Direct formation, no Google services
WP5	Relay and delay tolerant semantics	14	2,380	Small store and forward service, rotating mailboxes, quotas, eviction
WP6	Android reference client	26	4,420	Compose interface, join flow, sync status, accessibility, onboarding
WP7	Convergence and partition fuzzing	18	3,060	Simulator, schedule generator, differential oracle, energy rig
WP8	Field pilot, user research, documentation	17	2,890	A real deployment with a partner organisation, written up publicly
WP9	Community, upstream, F-Droid release	12	2,040	Upstream contributions, reproducible build, store release
	Total	**176**	**29,920**	Zero expenses

How the rate was derived

Senior Rust and Android engineer, Kathmandu	NPR 400,000 / month
In euro	2,350
Employer on costs, 28 percent	3,010
Studio overhead, 20 percent	3,770
Divided by 21.67 working days	174 / day
Rate we ask for	**170 / day**

This is cost recovery, not consultancy pricing. We ask slightly below our own derived cost.

> **Expenses: none.** Phones, laptops, hosting and compute are not eligible, so we request none. The studio already owns the test hardware. The security audit and the accessibility scan are free NLnet support services, so both appear as zero cost milestones rather than budget lines.

Two people at roughly a third of full time each over twelve months, about 0.68 of one full time person. The first two months are deliberately light because payment is in arrears and we carry that period ourselves.

Milestones and payment

Payment is per milestone, in arrears. The two zero cost gates are in the plan because the sample memorandum of understanding shows exactly these two, and because binding the release to passing them is the right incentive.

Gate	Name	When	Euro	Acceptance evidence
1	Foundations	Month 2	3,000	Threat model, wire spec, decision records public
2	Core and first transport	Month 5	7,000	CRDT core, identity, transport trait, stock Android path
3	Full transport matrix	Month 7	8,000	BLE discovery and Wi-Fi Direct on hardware without Google services
·	Accessibility gate	Month 9	0	WCAG audit through the free NLnet service, issues fixed
4	Product, pilot and evidence	Month 10	6,000	App complete, fuzzer green, pilot report published
·	Security gate	Month 11	0	External security review, issues triaged and resolved
5	Release and handover	Month 12	5,920	Reproducible build, F-Droid release, docs, upstream merged

6. Risk register

Six things that could go wrong, what each would cost, and what we would do. The honest position is that two of these could force a scope reduction. Saying so in advance is better than discovering it in month eight.

Risk	Level	Why it is real	Mitigation
Android background limits block the energy target	**High**	The operating system may simply forbid the background behaviour the design wants	Measure on named devices with two independent instruments. Publish the number achieved. If the target cannot be met, report it and narrow the claim rather than the measurement.
Wi-Fi Direct behaves differently across vendors	**High**	Group formation is known to vary on Huawei, Xiaomi, Samsung and Google builds	The device matrix is a deliverable, not an assumption. The client falls back to a plain socket over a hotspot when formation fails.
Free software radio path proves unworkable	**Medium**	The hardest engineering in the project	The transport trait preserves the library, relay and fuzzing deliverables if the radio work has to be cut. The de-scope ladder is published in advance.
CRDT growth over long partitions	**Medium**	Tombstones accumulate while replicas are apart for weeks	Compaction is tested against convergence, and a late joiner rebuilding from a snapshot is an explicit acceptance criterion.
Schema migration across app versions forks state	**Medium**	A replica on v2 receiving a v1 change must not diverge	Migration is written as a pure function and tested for commutativity with merge.
Scope creep into chat or maps	**Medium**	Both are obvious asks from users and both sink projects	The non goals are written down and published. Chat and maps stay out of the funded scope.

The de-scope ladder, published in advance

If the budget is	What changes	What is still delivered
about 30,000	Nothing is cut	Library, app, relay, fuzz kit, field pilot, all audits
about 25,000	Smaller pilot, less interface polish	All ship gates still met
about 20,000	No external pilot, BLE only, no Wi-Fi Direct bulk transfer	Library, app, relay, fuzzing harness, works on the measured device matrix
below 20,000	We would rather not apply than over promise	The scope stops supporting the impact case honestly

> **What would make us stop and say so.** If, after six weeks of measurement, idle mesh energy cannot get within a factor of two of the target on any device we own, we will report that in the milestone note and narrow the battery claim rather than quietly restate the number. The same applies to Wi-Fi Direct formation rates.

7. Prior art, stated plainly

MeshDoc is not the first mesh messenger, not the first CRDT library and not the first humanitarian data tool. The claim is narrower and more defensible: encrypted, signed, structured documents that converge over an intermittent serverless mesh, and that are released as a permissive library rather than as an app.

Project	What it is	Relationship to this work
Automerge, Yjs, Loro	CRDT libraries and sync engines	We use one of these rather than writing our own, cr-sqlite included. They deliberately leave identity, membership, encryption, discovery and relay to the application. That is the layer we build.
Briar	Messaging over Bluetooth, Wi-Fi and Tor	Carries messages. Its roadmap lists collaborative document editing as future work. Documents are the payload here.
Meshtastic	Text and position over LoRa	Different radio and a different payload. Interoperability is interesting later, not a competitor.
qaul, Berty, Irdest	Mesh and delay tolerant messaging or stack	Message or routing primitives. No typed document that merges.
KoboToolbox, ODK	Field data collection	The status quo, and server centric. Correct when a server exists. MeshDoc targets the case where it does not.
Keyhive	Group key agreement for local first data (NLnet funded)	Solves keys, not transport or a field toolkit. A future dependency candidate for group keys.
SecSync	Encrypted CRDT exchange (NLnet funded)	States the gap precisely. Server mediated, which is exactly what we are not.

What we will give back

Contribution	Why it is useful to others
Transport crate	BLE discovery and Wi-Fi Direct formation behind a narrow trait, usable by any local first project that needs peer discovery without Google services.
Partition fuzzing harness	Seeded edit and partition schedules with a differential oracle. Any CRDT backed project can point it at their document types.
Schema migration guidance	Written notes on making CRDT migrations commute with merge, which is the part most local first projects get wrong when they ship a second version.
Relay interoperability profile	A short document describing the mailbox format so other implementations can serve the same clients, rather than one binary becoming the de facto standard.

> **Why this belongs in the Open Internet Stack.** The commons already has excellent pieces for messages and excellent pieces for documents, but the document pieces are all server centric. What is missing is the middle layer that lets a document live on a phone and reach another phone without a server in between. That layer is infrastructure rather than a product, which is exactly the kind of thing that does not get built unless somebody is paid to build it.

Source code, tests and demonstration recordings: `https://github.com/sushant-me/meshdoc`

Licence: MIT. No contributor licence agreement, no dual licensing, no patent applications.